1. Project Title & Team Details

Project Title: Secure Capability-Based File System

Team Member: Yashika Dudeja - 24BCI0260

Guide/Faculty: Suchithra J

Course: Operating Systems
Department: Computer Science and Engineering (Information Security) , SCOPE

---

2. Introduction & Background

Introduction

Traditional file systems mainly use users, groups, and permission bits for resource protection. However, in a multi-process environment, these mechanisms may not provide sufficiently fine-grained control over which process can perform a specific operation on a particular resource.

The **Secure Capability-Based File System** addresses this by assigning explicit capabilities to processes. Each capability specifies the resource a process can access and the operation it is allowed to perform.

Context of the Problem

The project focuses on process-level access control for virtual files and directories. A capability is represented as:

Process + Resource + Allowed Operation

Every resource request is checked against the process's assigned capabilities before the operation is performed.

Relevance to Operating Systems

The project demonstrates:

- Process management
- File systems
- Protection and security
- Access control
- Resource management
- System calls
- Least privilege

---

3. Problem Statement

Problem Being Addressed

Traditional permission mechanisms may not provide sufficiently fine-grained control over process-level access to specific resources and operations.

The project addresses this by implementing capability-based access control in a virtual file system.

Existing Issue / Limitation

A process may require access to only specific resources and operations. Explicit capabilities allow these requirements to be represented more precisely than general user/group permissions.

Scope of the Project

The project is a simulation of relevant OS components and includes:

- Simulated processes
- Virtual files and directories
- Capability-based access control
- READ, WRITE, DELETE and EXECUTE operations
- Capability granting and revocation
- Access verification
- Audit logging

It does not attempt to implement a complete kernel or hardware-level operating system.

---

4. Objectives

Main Objective

To develop a mini operating-system file system simulator that provides fine-grained resource protection using capability-based access control.

Specific Objectives

- Develop a virtual file system supporting basic file and directory operations.
- Create and manage simulated processes with unique PIDs.
- Implement capability creation, storage and verification.
- Support READ, WRITE, DELETE and EXECUTE operations.
- Allow dynamic granting and revocation of capabilities.
- Maintain an audit log of successful and denied access attempts.
- Demonstrate OS concepts related to processes, file systems, protection and security.

Expected Outcome

The final system will allow processes to access resources only through explicitly assigned capabilities, reject unauthorized operations, support dynamic permission management, and maintain an audit trail.

---

5. Methodology / System Design

Proposed Approach

The project is implemented as a **C++ command-line file system simulator** consisting of five major modules:

1. **Process Manager** – Creates processes and assigns unique PIDs.
2. **Virtual File System** – Manages files, directories and file operations.
3. **Capability Manager** – Creates, stores, checks, grants and revokes capabilities.
4. **Access Controller** – Verifies capabilities before resource operations.
5. **Audit Logger** – Records successful and unauthorized access attempts.

These responsibilities follow the original system design.

System Access Flow

Process Request
↓
Access Controller
↓
Check Capability
↙ ↘
Found Not Found
↓ ↓
ALLOW DENY
↓ ↓
Execute Reject
↘ ↙
Audit Log

Technologies Used

Component	Technology
Language	C++
Interface	Command-Line
Compiler	GCC/G++
IDE	VS Code / C++ IDE
Data Structures	Classes, Structures, Vectors, Maps

OS Concepts Used

Processes, File Systems, Protection, Access Control, Resource Security, System Calls, Least Privilege.

---

6. Work Completed / Current Progress

The core system has been developed and the major modules have been implemented.

Activities Completed

- Finalized system architecture and project structure.
- Implemented simulated process management.
- Implemented the virtual file system.
- Implemented capability representation and storage.
- Implemented capability checking.
- Implemented the Access Controller.
- Added READ, WRITE, DELETE and EXECUTE operations.
- Implemented capability granting and revocation.
- Added audit logging.
- Integrated the major modules into the main workflow.

Module Status

Module	Status
Process Manager	Completed
Virtual File System	Completed
Capability Manager	Completed
Access Controller	Completed
Audit Logger	Completed
Module Integration	Mostly Completed
Comprehensive Testing	In Progress
Final Documentation	Pending

Current Project Status

Approximately 75% complete. The core functionality is in place, with comprehensive testing, edge-case handling, final refinements and documentation remaining.

---

7. Last Contributions / Recent Progress

Most Recent Work Completed

The latest work focused on integrating the individual modules so that a process request passes through capability verification before the requested operation is executed.

New Functionality Added

- Process creation with unique PIDs
- Virtual resource management
- Process-specific capabilities
- Capability verification
- READ, WRITE, DELETE and EXECUTE permissions
- Capability granting and revocation
- Access-denied handling
- Audit logging

Recent Improvement

The system was divided into independent modules, making the implementation easier to manage and allowing each OS concept to be demonstrated separately.

Current Achievements

The main access-control workflow is functional:

Process → Resource Request → Capability Verification → Allow/Deny → Operation → Audit Log

---

8. Remaining Work & Future Plan

Pending Tasks

- Complete module integration.
- Perform comprehensive testing.
- Test authorized and unauthorized operations.
- Test capability granting and revocation.
- Handle invalid PIDs, files, capabilities and operations.
- Improve command-line interaction.
- Debug and refine the integrated system.
- Complete final documentation and screenshots.

Testing / Validation

The system will be tested for:

- Authorized and unauthorized READ
- Authorized and unauthorized WRITE
- Authorized and unauthorized DELETE
- Authorized and unauthorized EXECUTE
- Capability granting
- Capability revocation
- Access after revocation
- Multiple processes and resources
- Invalid requests
- Correct audit-log generation

Expected Final Outcome

A functional command-line simulation of a capability-based file system where resource access is controlled through explicit process capabilities, with dynamic permission management and audit logging.